Cahier des Charges — Deep Learning for Spare Parts Management

Oct 28, 2026 · @Adam

1. Contexte et problématique

Le projet pilote conjointement la maintenance et le stock de pièces de rechange d'une flotte de machines, en remplaçant une politique classique à seuils fixes par une politique prédictive et décisionnelle fondée sur le deep learning.

Élément	Valeur
Intitulé	Deep Learning for Spare Parts Management
Encadrement	Pr. Aziza Belmaati, Pr. Oumaima Bounou
Cadre	ENSAM, semestre 9
Équipe projet	À compléter
Données	Microsoft Azure Predictive Maintenance (Kaggle), enrichi + données synthétiques du simulateur

Dans une flotte industrielle, la disponibilité dépend autant du stock que de la fiabilité : une panne sans pièce disponible transforme une réparation de quelques heures en arrêt de plusieurs jours. À l'inverse, un stock surdimensionné immobilise du capital. Les politiques classiques (seuil de commande fixe, maintenance préventive à intervalle fixe) ignorent l'état réel des machines et réagissent trop tard ou trop tôt.

Question centrale : une politique pilotée par deep learning (prédiction des pannes + agent d'apprentissage par renforcement) réduit-elle le coût total, à disponibilité égale ou supérieure, face à une politique à seuils fixes **correctement optimisée** ?

2. Objectifs

L'objectif général est de construire un jumeau numérique simplifié (simulateur) d'une flotte et de démontrer, chiffres à l'appui, le gain d'une politique deep learning sur une politique à seuils fixes.

#	Objectif spécifique	Livrable associé	Critère de réussite
O1	Préparer et enrichir le dataset Azure (durées d'étapes, régimes de maintenance, coûts, stock)	Dataset enrichi + dictionnaire de données	Schéma validé, aucune fuite temporelle
O2	Développer un simulateur à événements discrets (flux machine + flux stock + techniciens)	Package simulateur + tests	Reproduit les taux de panne Azure (écart < 10 %)
O3	Prédire les pannes et les besoins en pièces	Modèle prédictif entraîné	Rappel et F1 supérieurs à la baseline LightGBM
O4	Apprendre une politique de commande et de maintenance préventive par DRL	Agent entraîné	Coût total inférieur à la meilleure politique à seuils fixes, à disponibilité ≥
O5	Accélérer l'évaluation de scénarios par un modèle de substitution	Surrogate + analyse de sensibilité	$R^2 > 0,9$ sur scénarios de test, ×100 plus rapide que la simulation
O6	Comparer les politiques et restituer les résultats	Tableau de bord + rapport	Comparaisons avec intervalles de confiance à 95 %

Les seuils chiffrés (10 %, $R^2 > 0,9$, ×100) sont des propositions à valider avec l'encadrement.

3. Périmètre et hypothèses

Le projet couvre une flotte d'un site unique, un magasin de pièces unique et une équipe de techniciens partagée ; la chaîne d'approvisionnement amont est réduite à un délai fournisseur aléatoire.

Inclus

- Flotte de 100 machines (taille du dataset Azure), 4 composants remplaçables par machine (comp1 à comp4), donc 4 références de pièces.
- Scénario de référence S0 : machines identiques, conformément au sujet. Scénario étendu S1 : flotte hétérogène (4 modèles Azure, régimes de maintenance différenciés, voir section 6).

- États machine, flux stock, ressource techniciens, coûts, décisions de commande et de maintenance préventive.
- Commande normale et commande urgente (délai court, coût majoré).

Exclus

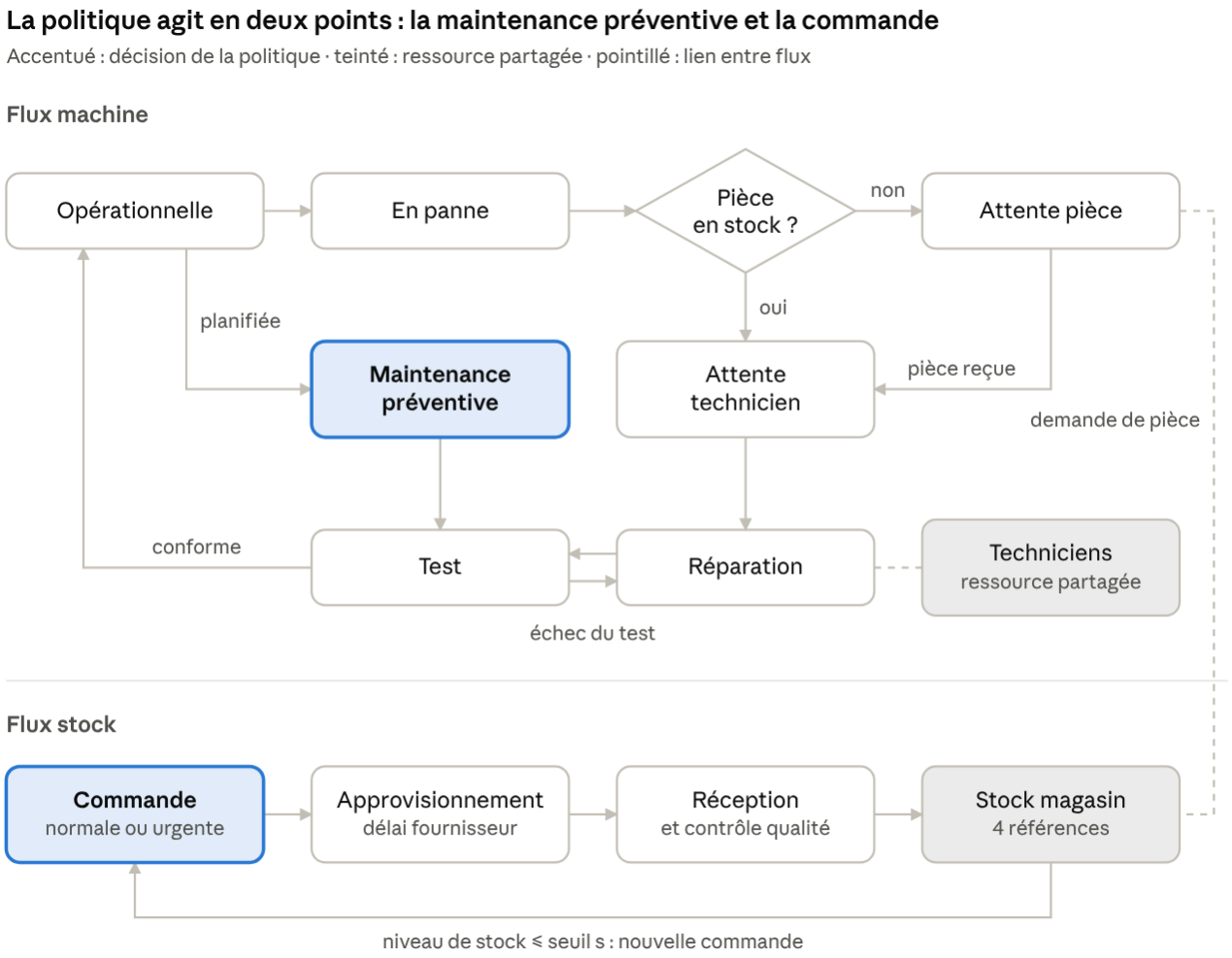
- Multi-sites, transferts latéraux entre magasins, réparation de pièces usagées (pièces réparables).
- Déploiement industriel réel et connexion à une GMAO.
- Ordonnancement détaillé de la production.

Hypothèses de modélisation

- Une panne consomme exactement une pièce du composant défaillant ; une maintenance préventive consomme aussi une pièce.
- Une pièce remplacée remet le composant à neuf (hypothèse « as good as new ») ; une variante « réparation imparfaite » pourra être testée.
- Les machines sont indépendantes sauf via les ressources partagées (stock, techniciens).
- Pas de cannibalisation de pièces entre machines.
- Horizon de décision : pas journalier (ou par quart de 8 h), résolution des événements à la minute.

4. Modélisation du système

Le modèle intègre le flux machine et le flux stock, reliés par la demande de pièce, et une ressource techniciens partagée ; la politique n'agit qu'en deux points, la maintenance préventive et la commande.



modèle intégré · flux machine, flux stock, techniciens

Une panne déclenche une demande de pièce : si le stock est vide, la machine attend la livraison. Par rapport au sujet, l'état « attente technicien » est ajouté explicitement, comme dans les articles de référence qui distinguent la disponibilité des techniciens. La maintenance préventive consomme elle aussi une pièce et un technicien.

5. Analyse du jeu de données Azure

Le dataset Microsoft Azure Predictive Maintenance fournit un an d'historique horaire (2015) sur 100 machines : il est riche pour la prédiction, mais ne contient ni durées d'intervention, ni stock, ni coûts, qu'il faut ajouter.

Fichier	Contenu	Volume	Usage dans le projet
PdM_telemetry.csv	volt, rotate, pressure, vibration (moyennes horaires)	876 100 lignes (100 machines × 8 761 h)	Entrées du modèle prédictif ; calibration du générateur de télémétrie

Fichier	Contenu	Volume	Usage dans le projet
PdM_errors.csv	errorID (error1 à error5), erreurs non bloquantes	≈ 3 900 lignes	Signes précurseurs de panne
PdM_maint.csv	comp (comp1 à comp4), remplacements planifiés et non planifiés, dès 2014	≈ 3 300 lignes	Âge des composants ; durées de vie censurées pour l'ajustement Weibull
PdM_failures.csv	failure (comp1 à comp4), remplacement suite à panne	761 lignes	Labels ; lois de panne ; demande de pièces
PdM_machines.csv	model (model1 à model4), age (années, 0 à 20)	100 lignes	Attributs machine ; régimes de maintenance

Les volumes de PdM_errors et PdM_maint sont approximatifs et seront confirmés lors de l'analyse exploratoire (EDA).

Limites à traiter

- Le dataset est lui-même synthétique (généré par Microsoft) : le projet est un démonstrateur méthodologique, pas une étude de cas réelle.
- Une panne et son remplacement portent le même horodatage : aucune durée de réparation, d'attente ou de test n'est observable.
- Aucune information de stock, de commande, de coût ou de technicien.
- Un seul an d'historique : les intervalles de 5 ou 10 ans ne sont pas observables directement (section 6).
- Pas de compteur de cycles : il sera dérivé du signal `rotate` (section 7).
- Fort déséquilibre : 761 pannes pour 876 100 heures-machine (≈ 0,09 %). Même avec un label « panne dans les 24 h », la classe positive reste autour de 2 % : l'accuracy est exclue au profit du rappel, du F1 et de la PR-AUC.

6. Types de machines et régimes de maintenance

L'intuition de départ est juste sur le fond, mais à corriger sur la forme : « tous les N cycles » et « tous les 5 ou 10 ans » sont deux variantes d'un même régime (la maintenance préventive systématique), qui s'applique à des composants et non à des machines, et il manque deux régimes essentiels.

La norme NF EN 13306 définit la maintenance préventive systématique comme exécutée « à des intervalles de temps préétablis ou selon un nombre défini d'unités d'usage ». Elle la distingue de la maintenance conditionnelle (déclenchée par l'état mesuré) et de la maintenance prévisionnelle (déclenchée par une prévision de l'évolution de l'état), qui est précisément ce qu'apporte le deep learning.

Ce qui est corrigé

1. Le régime dépend du mode de défaillance du composant, pas de la machine : une même machine combine souvent un composant suivi à l'usage et un autre au calendrier.
2. Le régime à l'usage (cycles, tours, heures de marche) convient aux dégradations par fatigue ou usure ; par exemple la durée de vie L10 d'un roulement s'exprime en millions de tours (ISO 281).
3. Le régime calendaire convient aux dégradations qui progressent même à l'arrêt : vieillissement des joints et élastomères, lubrifiants, isolants, corrosion, ainsi qu'aux contrôles réglementaires.
4. Des intervalles de 5 ou 10 ans correspondent à des révisions générales ou au remplacement de sous-ensembles majeurs. Pour des pièces de rechange courantes, les intervalles se comptent en semaines, en mois ou en milliers d'heures.
5. Avec un an de données, ces échéances longues sont modélisées via l'âge machine (0 à 20 ans dans Azure) : la prochaine révision tombe au prochain multiple de 5 ou 10 ans, et des simulations pluriannuelles couvrent les scénarios stratégiques.
6. Une maintenance préventive systématique n'est utile que si le taux de panne croît avec l'âge (paramètre de forme Weibull $\beta > 1$). Si $\beta \approx 1$ (pannes aléatoires), seul le stock peut améliorer la disponibilité ; ce critère sera vérifié composant par composant.

Régimes retenus

Régime	Déclencheur	Unité	Politique classique associée	Rôle dans le projet
Corrective	Panne	—	—	Toujours présente
R1 Systématique à l'usage	Compteur ⩾ N	Cycles, tours, heures de marche	Remplacement à l'âge (Age Replacement)	Baseline
R2 Systématique calendaire	Date ⩾ T	Jours, mois, années (révision 5 ou 10 ans)	Remplacement par bloc (Block Replacement)	Baseline

Régime	Déclencheur	Unité	Politique classique associée	Rôle dans le projet
R3 Conditionnelle	Signal mesuré > seuil fixe	Vibration, pression, tension…	Seuil d'alarme	Baseline renforcée
R4 Prévisionnelle	Risque prédit > seuil appris ou décision de l'agent	Probabilité de panne	Politique deep learning	Politique évaluée

Traduction dans le dataset

- Scénario S0 (machines identiques) : un seul profil de machine qui mélange les régimes par composant, par exemple deux composants R1 et deux composants R2.
- Scénario S1 (flotte hétérogène) : deux familles. Famille U « usage intensif », maintenue principalement en R1 ; famille C « vieillissement calendaire », maintenue principalement en R2 avec révision générale tous les 5 ou 10 ans.
- L'affectation de comp1 à comp4 et des modèles model1 à model4 aux régimes et familles sera décidée après l'EDA, à partir du signal précurseur de chaque composant et de son β Weibull. Les noms physiques donnés aux composants (roulement, joint…) resteront des hypothèses explicites, Azure ne les nommant pas.

7. Enrichissement du dataset

Le dataset enrichi sépare deux natures de durées : les durées intrinsèques (réparation, test, délai fournisseur) sont ajoutées comme lois paramétrées, tandis que les attentes (pièce, technicien) sont produites par le simulateur.

Cette séparation est un choix de conception central. Si l'attente de pièce était figée dans les données, elle ne dépendrait plus du stock et aucune politique, deep learning compris, ne pourrait la réduire.

7.1 Durées des étapes du processus

Étape	Nature	Loi proposée	Valeur indicative
Détection et diagnostic	Intrinsèque	Lognormale	Moyenne 1 h
Attente technicien	Endogène (simulateur)	—	Dépend de la charge

Étape	Nature	Loi proposée	Valeur indicative
Attente pièce	Endogène (simulateur)	—	0 si stock, sinon délai restant
Sortie magasin	Intrinsèque	Triangulaire	0,5 h
Réparation corrective	Intrinsèque, par composant	Lognormale	4 à 12 h
Maintenance préventive	Intrinsèque, par composant	Lognormale	≈ 60 % de la réparation corrective
Test après maintenance	Intrinsèque	Triangulaire + Bernoulli	1 à 2 h, échec 5 % puis retour en réparation
Traitement de commande	Intrinsèque	Constante	4 h
Délai fournisseur normal	Intrinsèque, par pièce	Gamma	7 à 21 jours
Délai commande urgente	Intrinsèque, par pièce	Gamma	1 à 3 jours, coût × 2 à 3
Réception et contrôle	Intrinsèque	Triangulaire	2 h
Révision générale (famille C)	Intrinsèque	Lognormale	2 à 5 jours

Les valeurs indicatives sont des hypothèses à justifier (littérature, avis des encadrantes). Leur influence sera mesurée par l'analyse de sensibilité (section 11).

7.2 Usage contre calendrier

Dans Azure, toutes les machines tournent 24 h/24 : le nombre de cycles est alors proportionnel au temps, et la distinction entre régimes R1 et R2 disparaît. On ajoute donc un calendrier de production par machine (1, 2 ou 3 postes de 8 h) : le vieillissement calendaire continue à l'arrêt, l'usure par l'usage non.

Le compteur d'usage est dérivé du signal `rotate` (interprété en tours par minute) : cumul horaire des tours depuis le dernier remplacement, exprimé en millions de tours, uniquement pendant les heures de marche.

7.3 Tables du modèle de données

Table	Grain	Colonnes principales	Origine
machines	1 ligne par machine	machineID, model, age_years, family (U/C), shifts_per_day, utilization_rate, next_overhaul_due, criticality, downtime_cost_per_h	Azure + ajouts
components	Modèle × composant	comp_id, regime (R1 à R3), pm_interval, pm_unit, weibull_beta, weibull_eta, precursor_signal, mttr_law, pm_duration_law, test_duration_law, test_fail_prob	Calibré sur Azure + hypothèses
parts	1 ligne par référence	part_id, unit_cost, holding_rate, order_fixed_cost, lead_time_law, expedite_lead_time_law, expedite_cost_factor, initial_stock, s, S	Hypothèses
technicians	Équipe	n_technicians, shift_calendar, hourly_cost, overtime_cost	Hypothèses
events	1 ligne par événement	run_id, policy_id, timestamp, machineID, comp, event_type, state_from, state_to, duration_h, part_id, qty, cost	Simulateur
state_intervals	1 ligne par séjour dans un état	run_id, machineID, state, start, end, duration_h, cause	Simulateur
ml_features	Machine × heure	Voir 7.4	Azure + simulateur

7.4 Variables ajoutées pour l'apprentissage

- Statistiques glissantes des 4 signaux (moyenne, écart-type) sur 3 h et 24 h.
- Nombre d'erreurs par type sur les 24 dernières heures.
- Par composant : temps depuis le dernier remplacement en heures calendaires, en heures de marche et en millions de tours.
- Par composant : distance à la prochaine échéance de maintenance préventive (cycles ou jours).
- Attributs machine : âge, modèle, famille, taux d'utilisation.

- Contexte logistique (dataset décisionnel) : stock disponible, position de stock (stock + commandes en cours), commandes en cours et leur date d'arrivée prévue.
- Labels : panne du composant k dans les 24 h, 48 h et 7 jours ; temps jusqu'à la panne, avec indicateur de censure.

8. Simulateur à événements discrets

Le simulateur est le socle du projet : il produit les données synthétiques, sert d'environnement d'entraînement à l'agent et arbitre la comparaison des politiques.

8.1 Composants du simulateur

- Processus machine : un processus par machine qui enchaîne les états de la section 4 et journalise chaque transition.
- Modèle de dégradation : pour chaque composant, un état de santé latent qui se dégrade avec l'usage (R1) ou le temps (R2). La panne survient selon une loi de Weibull calibrée sur Azure, ajustée par l'âge et le modèle.
- Générateur de télémétrie : niveau de base par modèle tiré des distributions Azure en fonctionnement normal, plus une dérive proportionnelle à la dégradation sur le signal précurseur, plus du bruit. Les erreurs non bloquantes sont tirées selon un processus de Poisson dont le taux croît avec la dégradation.
- Magasin : stock par référence, file des demandes en attente (premier arrivé, premier servi, ou par criticité), commandes en cours.
- Ressource techniciens : effectif limité, calendrier de postes ; une intervention mobilise un technicien pendant la réparation et le test.
- Interface de pilotage : à chaque époque de décision (chaque jour), le simulateur expose l'état et reçoit les décisions (commandes, maintenances préventives), puis avance jusqu'à l'époque suivante.

8.2 Calibration et validation

1. Ajuster une loi de Weibull par composant sur les durées entre remplacements Azure. Les remplacements planifiés (PdM_maint hors pannes) sont traités comme des observations censurées à droite, ce qui évite de sous-estimer la durée de vie.
2. Ajuster les covariables (modèle, âge) par un modèle de Weibull à temps accéléré (AFT).
3. Rejouer la politique observée dans Azure et comparer : nombre de pannes par composant et par modèle (écart cible < 10 %), distributions des signaux (test de Kolmogorov-Smirnov).
4. Test TSTR (Train on Synthetic, Test on Real) : un modèle prédictif entraîné sur données simulées doit garder une performance proche sur les données Azure réelles.

8.3 Exigences techniques

- Reproductibilité : graine aléatoire par réplication, nombres aléatoires communs (CRN) entre politiques comparées.
- Performance : une année simulée pour 100 machines en moins de 2 s, pour permettre des millions de pas d'entraînement.
- Période de préchauffage (warm-up) exclue des indicateurs.
- Paramètres externalisés dans des fichiers de configuration, aucun paramètre en dur dans le code.

9. Module prédictif (deep learning)

Recommandation : prédire, pour chaque composant, une courbe de risque sur plusieurs jours (modèle de survie à temps discret) plutôt qu'un simple « panne oui/non dans 24 h », car la décision de stock a besoin du risque sur tout le délai fournisseur.

9.1 Formulation

- Entrée : fenêtre glissante de 48 h des variables horaires (section 7.4) + attributs statiques de la machine.
- Sortie : pour chaque composant k et chaque jour j des 14 prochains jours, la probabilité de panne le jour j sachant la survie jusque-là. On en déduit la probabilité de panne sous 24 h (pour les métriques classiques) et sous le délai fournisseur L (pour le stock).
- Besoin en pièces : la demande attendue de la pièce k sur le délai L est la somme, sur les machines, des probabilités de panne sous L, plus les maintenances préventives planifiées. Sa distribution (Poisson-binomiale) donne directement un stock de sécurité par quantile.

9.2 Modèles comparés

Modèle	Famille	Atout	Limite	Rôle
Régression logistique	ML classique	Interprétable	Ignore la dynamique	Plancher
LightGBM sur variables glissantes	Gradient boosting	Très performant sur données tabulaires, rapide	Fenêtres choisies à la main	Baseline à battre
LSTM	Réseau récurrent	Demandé par le sujet, mémoire longue	Entraînement lent	Modèle DL de référence
GRU	Réseau récurrent	Plus léger que le LSTM, souvent équivalent	Mémoire un peu plus courte	Candidat principal

Modèle	Famille	Atout	Limite	Rôle
TCN (convolutions temporelles dilatées)	CNN 1D	Parallélisable, rapide, souvent au niveau ou au-dessus des RNN	Champ réceptif fixe	Candidat principal
Transformer (PatchTST, TFT)	Attention	Multi-horizon, interprétabilité (TFT)	Gourmand en données pour 761 pannes	Option si le temps le permet

Avec seulement 761 pannes réelles, les modèles légers (GRU, TCN) sont privilégiés ; les Transformers risquent le surapprentissage. Les deux architectures recevront la même tête de survie multi-composant.

9.3 Protocole

- Découpage temporel sans mélange : entraînement janvier à août, validation septembre-octobre, test novembre-décembre ; plus un test sur machines jamais vues.
- Déséquilibre : perte pondérée ou perte focale ; pas de suréchantillonnage synthétique type SMOTE sur des séries temporelles.
- Calibration des probabilités (temperature scaling ou régression isotone), indispensable puisque l'agent et le stock les utilisent ; contrôle par score de Brier et diagramme de fiabilité.
- Métriques : rappel, précision, F1 et PR-AUC par composant ; délai d'anticipation (heures entre la première alerte et la panne) ; indice de concordance (C-index) pour la courbe de survie.
- Deux jeux de résultats : sur Azure réel (résultats publiables) et sur données simulées (modèle embarqué dans le simulateur).
- Réglage des hyperparamètres par Optuna.

10. Agent d'apprentissage par renforcement profond

Recommandation : PPO avec masquage d'actions (MaskablePPO) comme agent principal, et un DQN à actions factorisées (Branching Dueling Q-Network) comme comparaison. Un DQN standard est inadapté ici : il doit énumérer toutes les combinaisons d'actions, et ce nombre explose avec la taille de la flotte.

10.1 Formulation (processus de décision markovien)

- Époque de décision : chaque jour, soit 365 pas par épisode d'un an.

- Observation : par pièce, stock disponible, commandes en cours et jours avant arrivée, demandes en attente ; par machine, état courant, temps depuis remplacement de chaque composant, distance à l'échéance de maintenance préventive, risque prédit par le module DL (sous 24 h et sous le délai fournisseur) ; globalement, techniciens disponibles et jour de l'année.
- Récompense : opposé du coût du jour, qui additionne stockage, commandes (fixe + unitaire), surcoût des commandes urgentes, maintenances préventives et correctives, heures d'arrêt machine valorisées, pénalité par heure d'attente de pièce et heures supplémentaires des techniciens. La disponibilité entre dans la récompense via le coût d'arrêt ; une contrainte explicite de disponibilité minimale (RL contraint, méthode lagrangienne) est une option.

10.2 Deux niveaux d'action

Niveau	Ce que l'agent décide	Taille de l'espace	Atout	Risque
N1 Paramétrique (premier livrable)	Chaque jour, par composant : seuil de risque déclenchant la maintenance préventive et niveau de recomplètement S du stock ; une règle exécute	Petit (quelques niveaux par composant)	Apprentissage stable, politique lisible	Gain plafonné par la règle
N2 Direct (objectif ambitieux)	Quantité à commander par pièce, commande urgente oui/non, maintenance préventive machine par machine	Très grand, multi-discret	Politique la plus fine	Apprentissage plus long, nécessite le masquage

Le masquage interdit les actions impossibles : maintenance d'une machine déjà en panne ou en réparation, maintenance sans pièce en stock ou sans technicien libre.

10.3 Algorithmes comparés

Algorithme	Famille	Actions multi-discrètes	Rôle
MaskablePPO	Gradient de politique + masques	Oui, natif	Agent principal (N1 et N2)
PPO	Gradient de politique	Oui, natif	Ablation : mesure l'apport du masquage
DQN (Double + Dueling)	Fonction de valeur	Non, sauf espace réduit	Comparaison sur N1 réduit, demandé par le sujet
Branching Dueling Q-Network	Fonction de valeur factorisée	Oui, une branche par dimension	Comparaison sur N2
RecurrentPPO (PPO + LSTM)	Gradient de politique récurrent	Oui	Option si l'observabilité partielle pénalise l'agent
MAPPO (un agent par machine + un agent stock)	Multi-agent	Oui	Perspective, hors périmètre principal

10.4 Entraînement

- Environnements vectorisés (8 à 16 simulateurs en parallèle) et normalisation des récompenses.
- Curriculum : d'abord 10 machines, puis la flotte complète de 100 machines.
- Évaluation périodique sur des graines fixes, avec nombres aléatoires communs face aux baselines.
- Plusieurs graines d'entraînement (au moins 3) pour rapporter la variabilité de l'agent, pas un seul meilleur run.

11. Modèle de substitution (surrogate)

Le modèle de substitution apprend la relation « paramètres d'un scénario → indicateurs de performance » à partir de quelques milliers de simulations, pour répondre en quelques millisecondes à une question de type « que se passe-t-il si… ».

Aspect	Choix proposé
Entrées	Paramètres de politique (s et S par pièce, intervalles de maintenance préventive, seuil de risque) et du système (nombre de techniciens, délai fournisseur moyen, taille de flotte, multiplicateur des taux de panne, coûts)
Sorties	Disponibilité, attente moyenne de pièce, fréquence de ruptures, coût total et sa décomposition
Plan d'expériences	Hypercube latin, 2 000 à 5 000 scénarios, 5 réplications chacun
Modèle principal	Ensemble de 5 réseaux MLP (deep ensemble) : prédiction + incertitude
Modèles de comparaison	Processus gaussien (référence pour petits budgets), LightGBM
Validation	R^2 et erreur relative moyenne (MAPE) sur scénarios jamais vus ; tout optimum trouvé par le surrogate est revérifié par simulation

Trois usages

1. Optimiser la politique à seuils fixes par optimisation bayésienne, pour que le deep learning soit comparé à la meilleure baseline possible et non à une baseline mal réglée.
2. Analyse de sensibilité globale (indices de Sobol) sur les hypothèses de durées et de coûts de la section 7.1.
3. Simulation instantanée dans le tableau de bord (curseurs → indicateurs en temps réel).

Perspective : un modèle appris de la dynamique du simulateur (world model) permettrait un apprentissage par renforcement basé modèle, hors périmètre principal.

12. Politiques comparées et protocole d'évaluation

La comparaison oppose six politiques sur les mêmes scénarios aléatoires ; la politique à seuils fixes est optimisée avant comparaison, sinon le gain du deep learning serait surestimé.

12.1 Politiques

Code	Politique	Stock	Maintenance préventive	Ce qu'elle mesure
P0	Seuils fixes « métier » (sujet)	(s, S) réglé à la main	Systématique R1/R2, intervalles constructeur	Point de départ
P1	Seuils fixes optimisés	(s, S) optimisé	Systématique, intervalles optimisés	Baseline principale
P2	Conditionnelle	(s, S) optimisé	Seuil fixe sur signal mesuré	Apport de la surveillance seule
P3	Prédictive heuristique	Recomplètement selon la demande prédite	Risque prédit > seuil fixe	Apport de la prédiction sans RL
P4	DRL niveau N1	Choisi par l'agent	Seuil de risque choisi par l'agent	Apport du RL
P5	DRL niveau N2	Choisi par l'agent	Choisie par l'agent	Apport du RL direct
Oracle	Connaissance parfaite des pannes futures	Idéale	Idéale	Borne supérieure atteignable

12.2 Indicateurs

Indicateur	Définition	Unité
Disponibilité	Temps opérationnel / temps d'ouverture	%
Attente de pièce	Temps moyen passé en attente de pièce par intervention	h
Fréquence de ruptures	Demandes non servies immédiatement / demandes totales	%
Coût total	Stockage + commandes + urgences + maintenance préventive + corrective + arrêts	UM par an
Stock immobilisé moyen	Valeur moyenne du stock	UM
MTBF, MTTR	Temps moyen entre pannes, temps moyen de réparation	h

Indicateur	Définition	Unité
Qualité de prédiction	Rappel, précision, F1, PR-AUC, délai d'anticipation	—

UM = unité monétaire, à fixer (MAD ou euro).

12.3 Protocole statistique

- 30 réplications d'un an par politique, nombres aléatoires communs, intervalles de confiance à 95 %.
- Tests appariés (Wilcoxon) entre P1 et chaque politique deep learning.
- Front de Pareto coût total / disponibilité pour visualiser les compromis.
- Robustesse : agent entraîné sur le scénario nominal puis testé avec délai fournisseur × 2, taux de panne + 30 %, un technicien de moins, et sur la flotte hétérogène S1.

13. Stack technique recommandée

Recommandation : un écosystème 100 % Python autour de SimPy, Gymnasium, PyTorch et Stable-Baselines3, qui couvre simulation, deep learning et RL sans glue entre langages et tourne entièrement sur CPU.

Couche	Recommandation	Alternative	Justification
Langage	Python 3.11	—	Compatible avec toutes les bibliothèques ci-dessous
Environnement	uv (fichier de verrouillage)	conda, pip + venv	Installation rapide et reproductible
Données	pandas + Parquet (PyArrow)	Polars	Standard, Parquet 5 à 10 fois plus compact que le CSV
Simulation	SimPy	Salabim ; moteur maison (file de priorité) si SimPy est trop lent	Référence de la simulation à événements discrets en Python, ressources et files natives
Interface RL	Gymnasium	—	Standard actuel des environnements RL

Couche	Recommandation	Alternative	Justification
Apprentissage par renforcement	Stable-Baselines3 + sb3-contrib	CleanRL ; RLlib	MaskablePPO, RecurrentPPO, DQN prêts et testés ; le Branching Q-Network sera codé en PyTorch
Deep learning	PyTorch	TensorFlow / Keras	Même framework que SB3, flexible pour la tête de survie
ML classique et fiabilité	scikit-learn, LightGBM, lifelines	XGBoost	Baselines, Weibull et modèle AFT avec censure
Optimisation	Optuna	Ax / BoTorch	Hyperparamètres et optimisation de la baseline P1
Plan d'expériences et sensibilité	SciPy (hypercube latin), SALib	—	Indices de Sobol
Suivi d'expériences	MLflow local + TensorBoard	Weights & Biases	Gratuit, hors ligne, comparaison des runs
Configuration	Hydra (YAML)	pydantic-settings	Scénarios et balayages de paramètres sans toucher au code
Tableau de bord	Streamlit + Plotly	Dash ; React + FastAPI	Interface en Python pur, livrable rapidement
Qualité	pytest, ruff, pre-commit, GitHub Actions	—	Tests du simulateur indispensables
Rapport	LaTeX (Overleaf)	Word	Formules et bibliographie

Matériel : tout fonctionne sur CPU ; un GPU (local, Google Colab ou Kaggle) accélère seulement l'entraînement des modèles prédictifs.

Organisation du dépôt

```
Deep-Learning-for-Spare-Parts-Management/
├── configs/              # scénarios, coûts, lois de durées (YAML)
├── data/
│   ├── raw/              # CSV Azure (non versionnés)
│   └── processed/        # tables enrichies (Parquet)
├── notebooks/            # EDA, calibration, analyses
├── src/spareparts/
│   ├── data/             # chargement, enrichissement, variables
│   ├── reliability/      # Weibull, AFT, calibration
│   ├── sim/              # simulateur SimPy
│   ├── envs/             # environnement Gymnasium
│   ├── policies/         # P0 à P3 et oracle
│   ├── predict/          # LightGBM, LSTM, GRU, TCN
│   ├── rl/               # MaskablePPO, DQN, Branching Q-Network
│   ├── surrogate/        # deep ensemble, processus gaussien, Sobol
│   └── eval/             # indicateurs, protocole, tests statistiques
├── app/                  # tableau de bord Streamlit
├── tests/
└── reports/              # figures et rapport
```

14. Livrables et planning

Le projet se découpe en sept lots livrables dans l'ordre ; les durées ci-dessous sont relatives (en semaines) et seront calées sur l'échéance réelle une fois celle-ci confirmée.

Lot	Contenu	Livrable	Durée indicative	Jalon de validation
L0	Cadrage	Cahier des charges validé	1 semaine	Validation par l'encadrement
L1	EDA, calibration Weibull, enrichissement	Dataset enrichi + dictionnaire de données + notebook EDA	2 semaines	Régimes et familles fixés (section 6)
L2	Simulateur + politiques P0 à P2	Package simulateur testé	2 à 3 semaines	Taux de panne Azure reproduits (écart < 10 %)
L3	Module prédictif	Modèles LightGBM, LSTM, GRU, TCN + rapport de métriques	2 semaines (en parallèle de L2)	F1 supérieur à LightGBM

Lot	Contenu	Livrable	Durée indicative	Jalon de validation
L4	Agent DRL (N1 puis N2) + P3	Agents entraînés + courbes d'apprentissage	3 semaines	Coût inférieur à P1 sur le scénario nominal
L5	Surrogate + sensibilité + optimisation de P1	Surrogate validé + indices de Sobol	1 à 2 semaines	$R^2 > 0,9$
L6	Évaluation finale, tableau de bord, rapport, soutenance	Application Streamlit + rapport + présentation	2 semaines	Soutenance

L5 doit précéder la comparaison finale : c'est lui qui optimise la baseline P1.

15. Risques et parades

Risque	Probabilité	Impact	Parade
L'agent DRL ne bat pas la baseline optimisée	Moyenne	Élevé	Niveau N1 d'abord ; récompense normalisée ; un résultat négatif bien expliqué reste un résultat
Simulateur trop lent pour l'entraînement RL	Moyenne	Élevé	Profilage dès L2 ; pas journalier ; environnements parallèles ; moteur maison si besoin
Données synthétiques trop faciles (prédiction quasi parfaite)	Moyenne	Moyen	Bruit et variabilité inter-machines ; test TSTR sur Azure réel
Hypothèses de durées et de coûts contestées	Moyenne	Moyen	Hypothèses documentées et justifiées ; analyse de sensibilité (Sobol)
Fuite d'information dans le module prédictif	Faible	Élevé	Découpage strictement temporel ; variables calculées uniquement sur le passé
Dérive du périmètre (multi-agent, Transformers…)	Élevée	Moyen	Options placées explicitement en perspective

16. Points ouverts à valider

- [ ] Composition de l'équipe et répartition des lots
- [ ] Échéance de soutenance et jalons intermédiaires imposés
- [ ] Articles de référence fournis par l'encadrement : à intégrer pour aligner les états et les notations
- [ ] Scénario S0 seul (machines identiques) ou S0 + S1 (flotte hétérogène)
- [ ] Stack technique (section 13) et outil de restitution
- [ ] Unité monétaire et ordres de grandeur des coûts
- [ ] Seuils de réussite de la section 2

Sources

- Microsoft Azure Predictive Maintenance, Kaggle
- H1st, tutoriel sur le dataset Azure (volumes de données)
- NF EN 13306, Maintenance — Terminologie de la maintenance
- ISO 281, Roulements — Charges dynamiques de base et durée nominale